



CS 522 – Selected Topics in CS

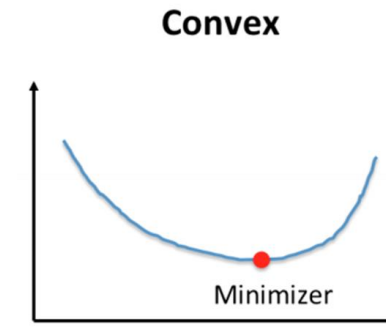
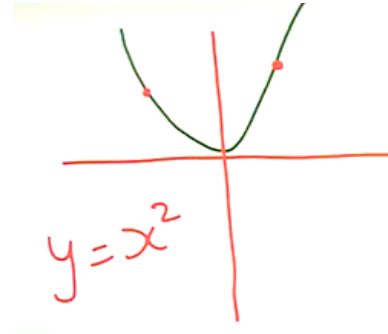
Lecture 08 – Convolutional Neural Network

+ Outline

- Base concepts
- Optimizers
 - Gradient Decent
 - Batch Normalization
 - Dropout
- Convolutional NN
 - Basic idea
 - Pooling Layer
 - CNN modeling
 - Batch Size
 - Epoch
- Transfer Learning
- Recurrent NN

+ Convex & Non-Convex Functions

- What's Convex and non-convex functions and how they are related to Deep Neural Network?



- **Convex Function**

- Let's take an example:

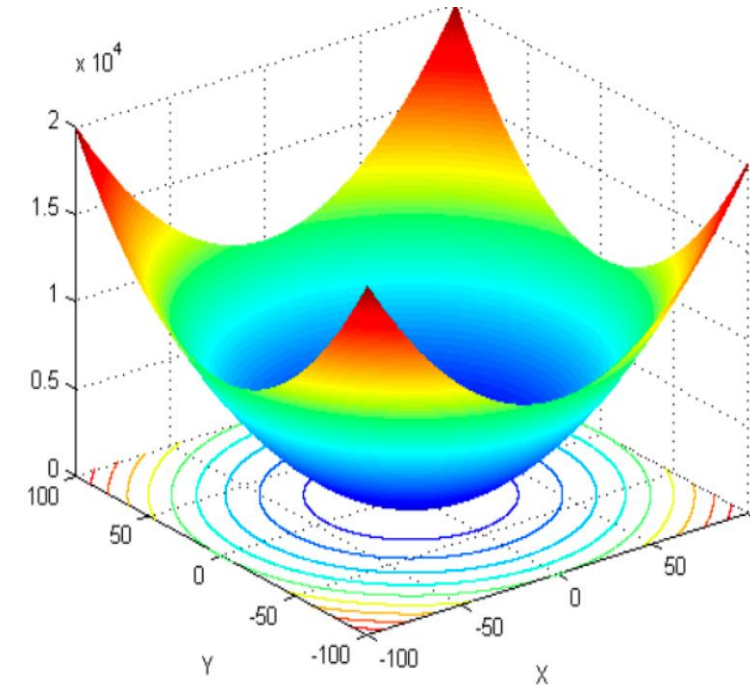
- If we draw the graph from expression, $y = x^2$, we will get the graph in OVAL shape, then it is called **CONVEX Function**.
- If we take any 2 points on this plane except the minimum point, we got the above the functions.
- For Convex function, we have 1 minimum point called **global minimum point**.

For the convex Function there will always be one variable Equation.

+ Convex & Non-Convex Functions

■ Contour-Line

- If we take convex function with **multi-variables** and if we take any points with similar heights on this graph, it will sketch one round circle on the surface of convex function.
- These circle are called **Counter Line**.
- The counter-line are used to visualize the 3D or 2 variables convex function in a 2D plan using circle.
- It also, allow us to **visualize exact point** of **global minima**

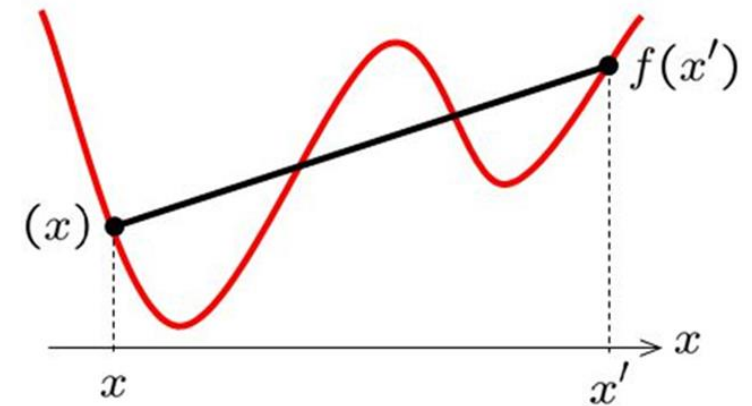
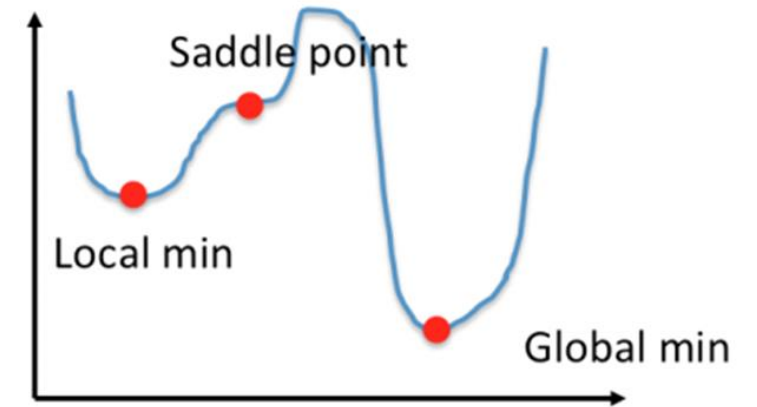


+ Convex & Non-Convex Functions

■ Non-Convex Function

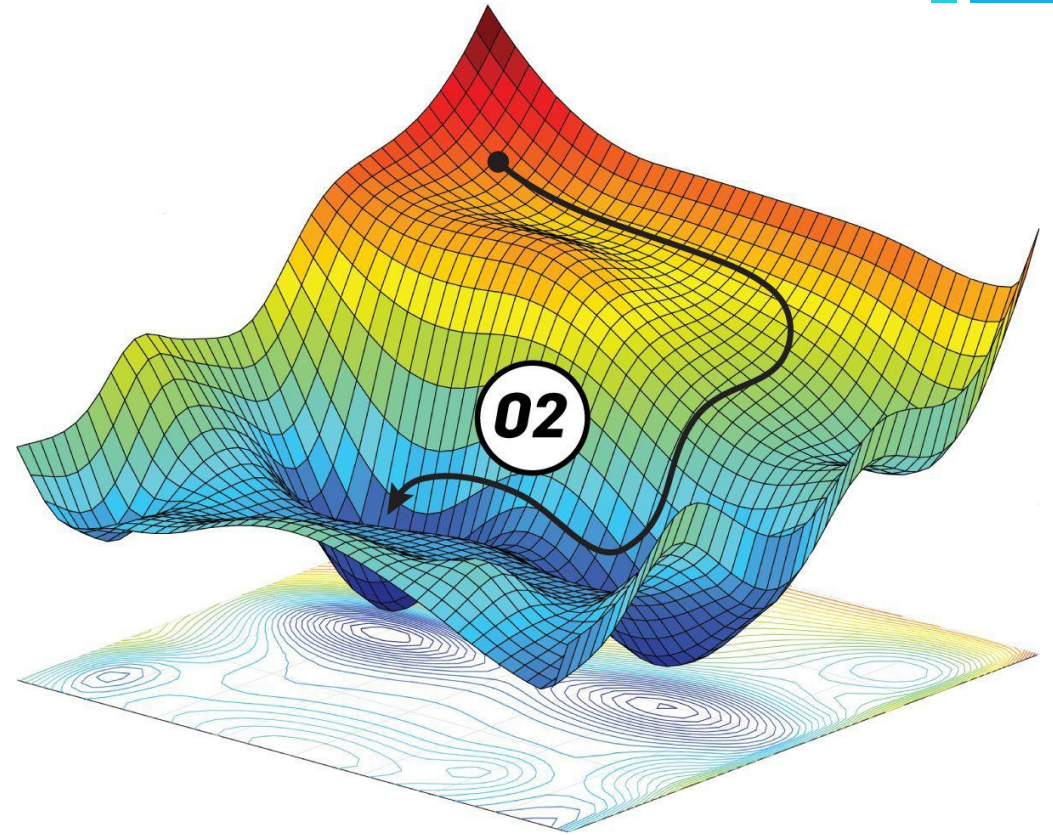
- In the non-convex function, there's no straight line, nor it is a smooth curve.
- It is curve with multiple ups and down
- This kind of functions are called **Non-Convex Function**

Non-Convex



+ Convex & Non-Convex Functions

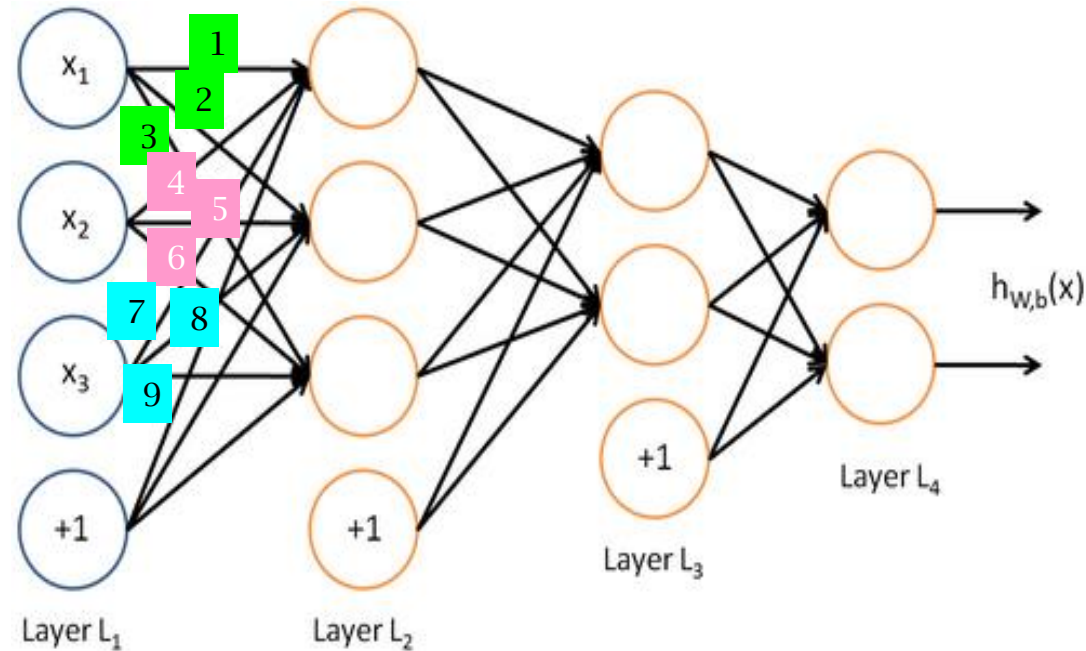
- **Non-Convex Function**
- Contour line give us clear view that where is the minimal local or global minima exist.
- Now Let's map these Functions with Deep Neural Network.



+ Deep Neural Network & Non-Convex Function

- DNN models for regression model are using loss functions (mean-square-error, Absolute-Mean-Square error etc.).
- If fact, these loss functions are 1-variable convex function and has one minimal point (also called global minima).
- With classification, we are using Non-Convex Function approach, because we have more than 1-variables and for that variables, we are finding out the global minimal point that have minimum error value for Loss function.
- Let's take an example, where we will try to find that the neural network is working with a Non-Convex function.

+ Deep Neural Network & Non-Convex Function



- The NN has some inputs, hidden layers and output layer. The **weights** are initialized with **random values**.
- The first neuron have W 's = 1, 4, 7, while second neuron has W 's = 2, 5, 8 etc. and third neuron has W 's = 3, 6, 9.
- It means there are **different local points** (weights) for the value that we need to find out.

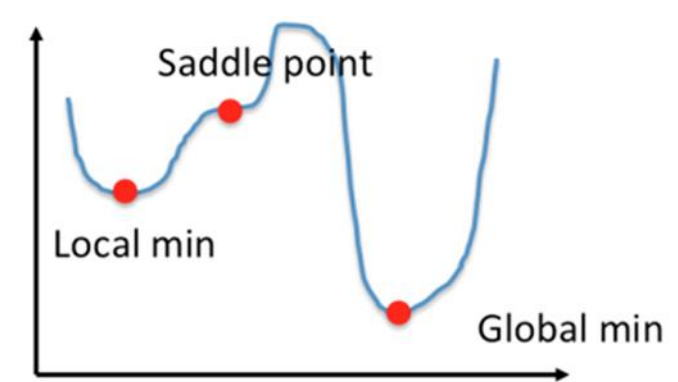
+ Deep Neural Network & Non-Convex Function

- Now, if we exchange (**SWAP**) the W's assign to neuron. We will get the same values.
- Means we are getting **same value** for **different possibilities** of the weights assign to NN, and we are getting almost **same global minima**.
- With **multiple W's**, we have **multiple local minima's** and one of them is global minima.
- By using **Backpropagation** technique, we **updates the W's** values and **calculate the LOSS** function of the model.
- In fact, we are **trying to find** the **Global Minima** throughout the process.

+ Deep Neural Network & Non-Convex Function

- **Questions:** How we can find out the global minima, if more than one minimal point exists OR How we can make sure that we are finding the correct minimal (global minima) point?
- **Answer:** By using **Backpropagation and SGD**, we are trying to **calculate** and update the **W's value** and then trying to find the **minimal point** (expected to be **global minimal point**).
- But sometimes, it happen that we can get stuck with the **saddle point** (which is one of the biggest problem with Non-Convex function).

+ Saddle Point

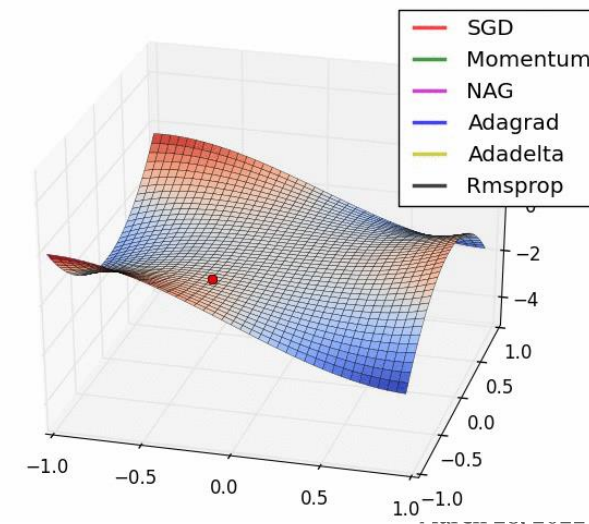


11

- Before discussing about the Saddle point, let's have an example.
- In the fig. there's a **small curve** having **saddle** point.
- Usually, the saddle points on the function are **linear functions**, and we know that if we take **derivative** of **linear** function, we are getting **constant value**.
- Means, the saddle point are the points on the function, **that are not changing their positions** (they are stuck points) and not allowing us to move either to maximal or minimal point.
- Taking **differentiations** of the **saddle** points, we are getting **answer 0**, means that there's **no learning** occur and there's **no changes for the model to learn**

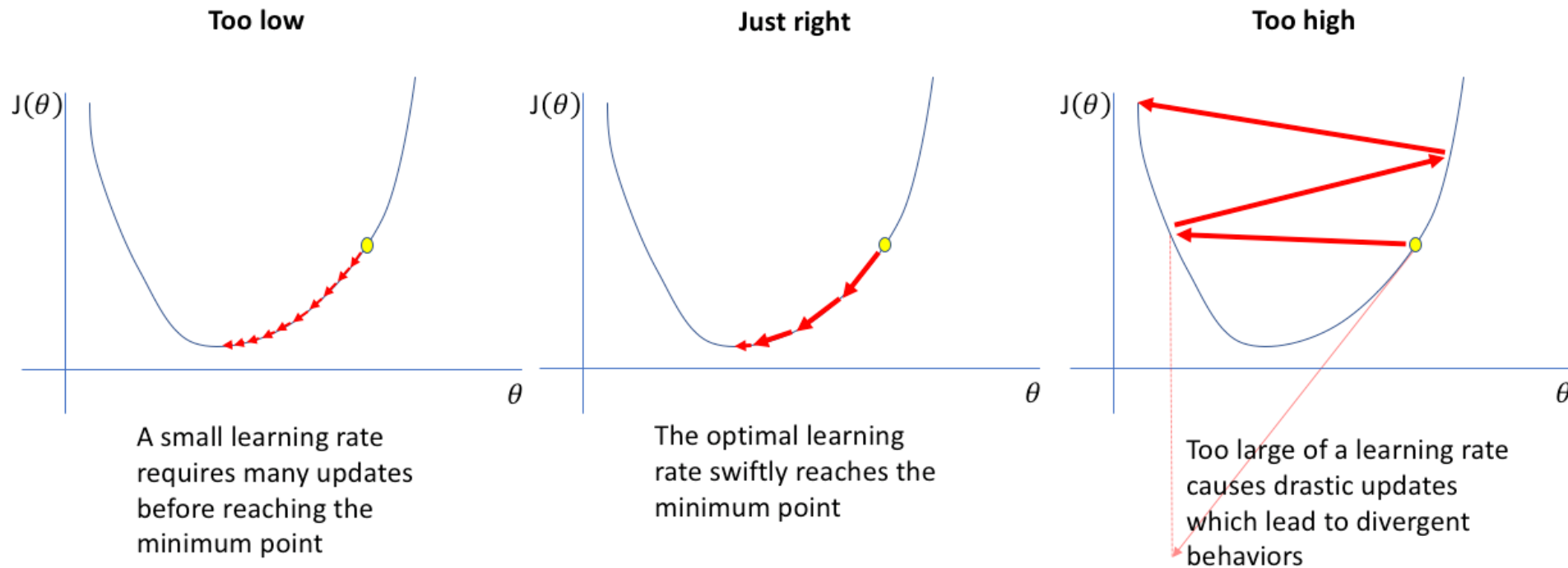
+ Problems with Saddle Point

- As with Backpropagation, we are calculating **LOSS** and by using **SGD** updates the **W's** and then we are getting **good results**.
- If the model is stuck at some point, we called it **SADDLE** Point and the model is not getting further **tune-up** (means not getting new values for **W's**), as a result the **model** is **not learning** from the data.
- This situation is called model is stuck with Saddle Point.
- Here we need to add the **Learning Rate**.



+ Learning Rate

- the learning rate is a **tuning parameter** in an **optimization algorithm** that determines the **step size** at each **iteration** while moving toward a **minimum** of a **loss function**.
- Learning Rate should be **between 0-1**.

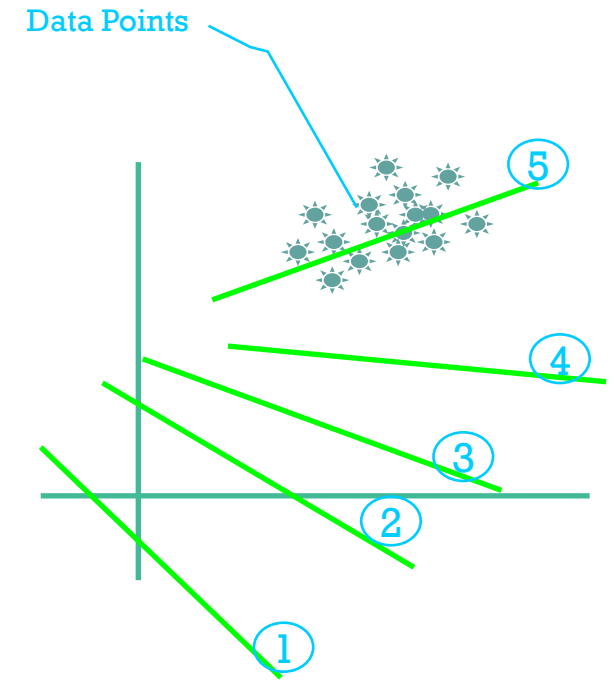




Gradient Descent Algorithms

+ Process of Gradient Decent Algorithm

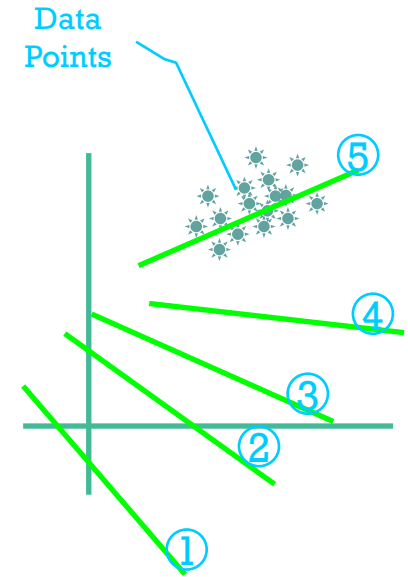
- Let's understand this problem related to SGD with example of using **Linear Regression**
- With initial **random assign values** to W 's, we get line (1) line, after 1st iterations and update W 's values, we get line (2) and so on.
- To update the W 's values, we have option to use **Gradient Decent**, **Stochastic Gradient Decent** and **Batch-Gradient Decent** algorithms.



+ Process of Gradient Decent Algorithm

- W's value are updated using

$$\blacksquare W_{1-new} = W_{1-old} - \alpha \sum_{i=1}^n \frac{\partial L}{\partial W_1}$$

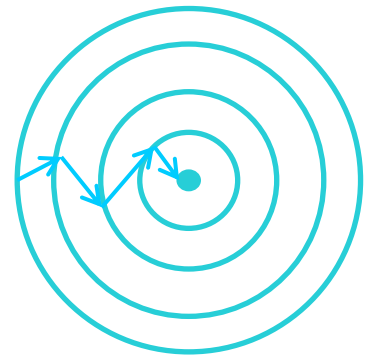
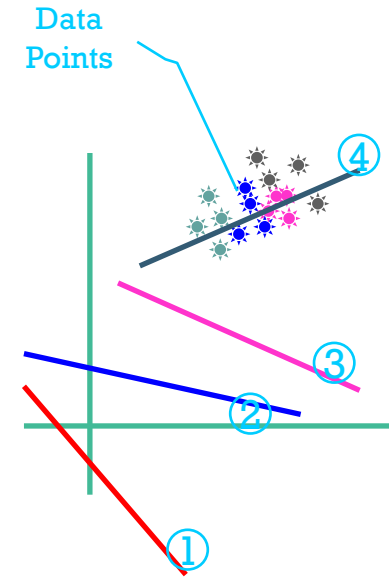


Important Point:

- Based on all points (data points), the w's are updated using GD-Algorithm.
- Advantages of GD:
 - **Convergence Fast:** It has a fast convergence towards minimal point
- Disadvantages of GD:
 - **Computational Costly:** As it consider all data points for derivative and then sum all derivative values, that's why it's computationally costly.

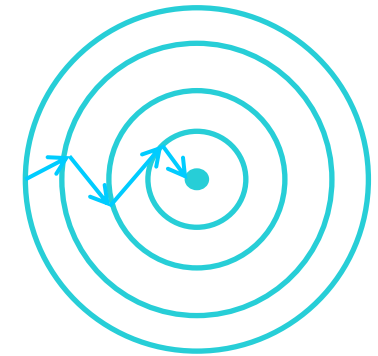
+ Process of SGD & BGD

- **SGD**: Taken **each points** and updates **W's values**.
- **BGD**: Taken **batch/group** of data points and **updates W's values**.
 - Let's take 4-batch size, in total we have 4-sets
 - With initial values, we are getting line-(1)
 - With 1st batch (red-points), we are getting line-(2)
 - With 2nd batch (blue-point), we are getting line-(3)
 - and so on... finally got full convergence (minimal point)
- Linear-Regression are taken **different directions**, but finally it reach to the **convergence**. Let's understand this by Contour-line.



Contour-line.

+ Process of SGD & BGD

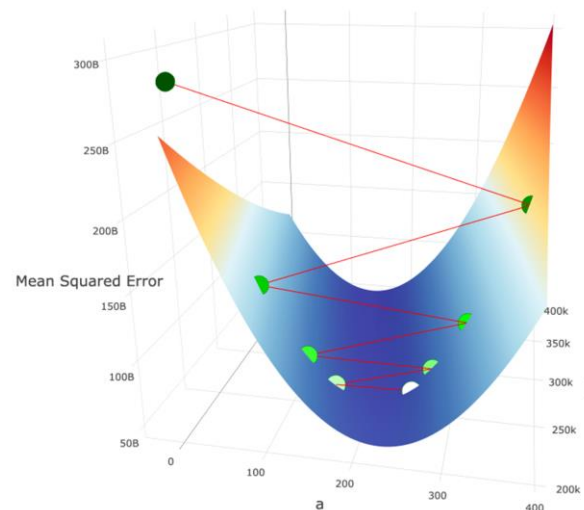


■ Advantages of SGD/BGD:

- **Computational Easy:** As it consider less data points for derivative and then **sum all derivative values**, that's why it's **computationally** easy.

■ Disadvantages of SGD/BGD:

- **Convergence Slow:** It has a **slow** convergence towards **minimal point**



+ Application of GD, SGD, BGD in NN

- If we take a **big** NN, means **multiple layers and multiple neurons**. So **how many W's will be there**.
- To obtain the minimum loss – mean to update the W's values to get convergence. It will take a lot of time.
 - If we use GD, it have fast convergence, but **computationally costly**.
 - If we use SGD/BGD, it have **SLOW-convergence**, but computationally **easy**.

What's the solution:

- We need to follow **SGD or BGD** but also, we need to improve **convergence speed**.

How to improve the Convergence Speed: We need to **adopt Optimizers**.

+ Optimization

- **How to deal with the Disadvantages of SGD/BGD**
- There are various optimizer that could support us in getting optimal outcomes.
 - Ada Grad & Ada Delta Optimizers
 - Adam etc.



Batch Normalization

Documentation https://keras.io/api/layers/normalization_layers/

Example <https://machinelearningmastery.com/how-to-accelerate-learning-of-deep-neural-networks-with-batch-normalization/>

+ Batch Normalization

■ Before discussing **Batch-Normalization**, let's take an example,

Cat



Non-Cat

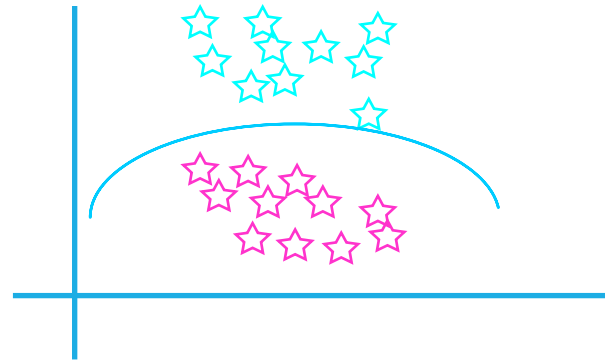
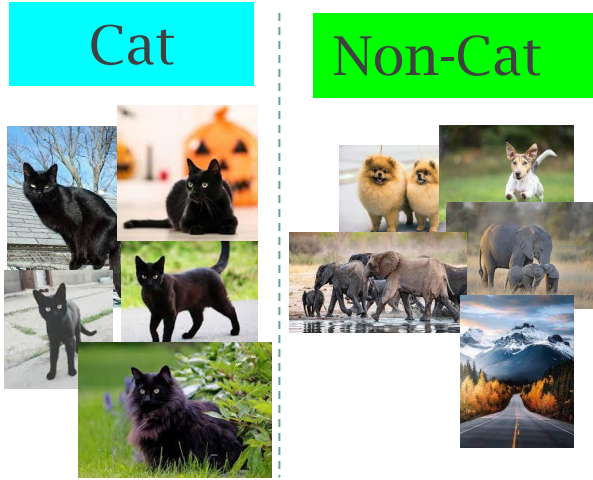


- Suppose, we are taking images of cat and non-cat to classify by DNN.
- Suppose, if we take black-cat images for **TRAINING** and classification purpose

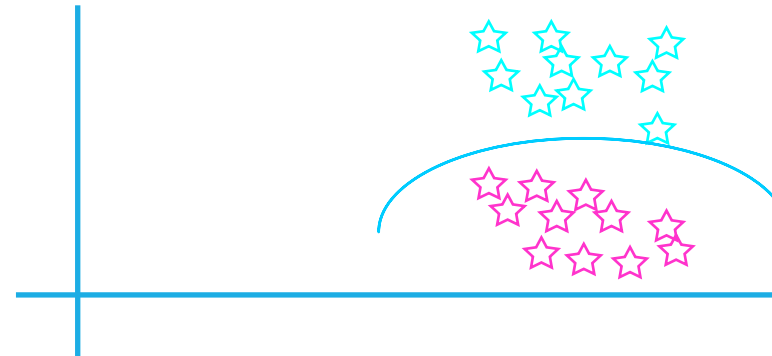
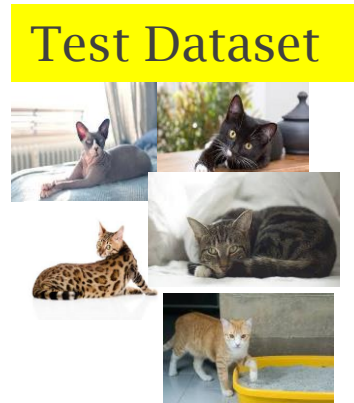
+ Batch Normalization

■ Before discussing Batch-Normalization, let's take an example,

■ If we distribute the **Training** images data, it will look like this



■ If we distribute the **Testing** images data, it will look like this



■ It will be a little bit on the **right side**, because the images have **different** colors representing cat

+ Batch Normalization

- *Problem: If the input distribution during TRAINING is change from the distribution of TEST data. Then the model will be unable to classify the data properly.*
- **Solution:** Apply BatchNormalization to each layer to Normalize them, i.e., adding a sub-layer within hidden layer with name BatchNormalization()

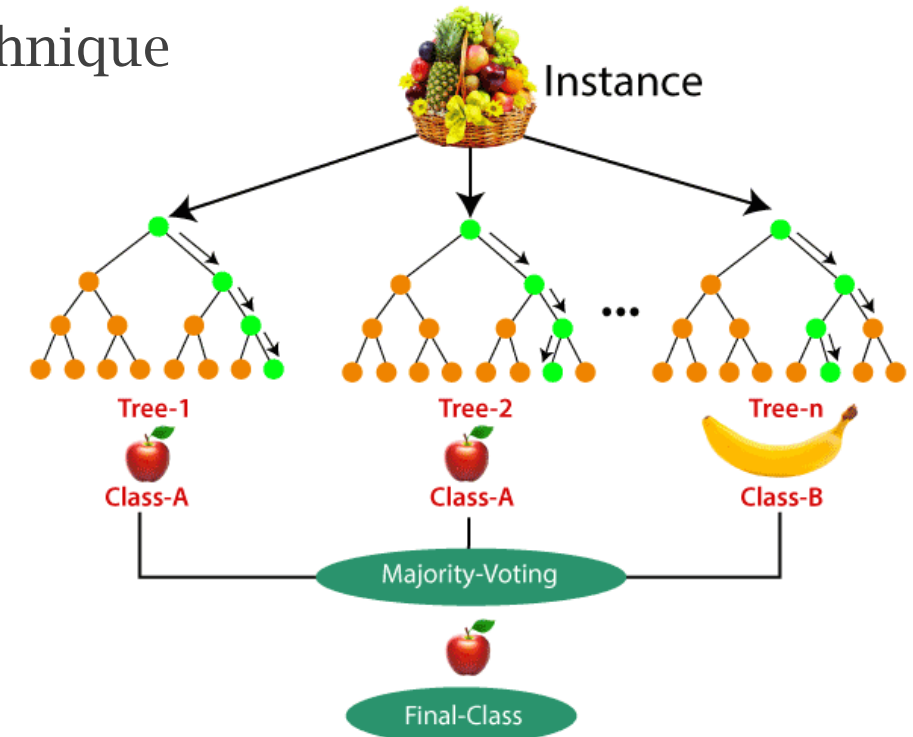
Dropout

Documentation https://keras.io/api/layers/regularization_layers/dropout/

Example <https://machinelearningmastery.com/dropout-regularization-deep-learning-models-keras/>

+ Dropout

- **Dropout:** Dropout is a technique to avoid overfitting.
- But before going to Dropout, let's discuss about the Ensemble technique and then we will go Neural Network.
- The **Random Forest classifier** is using Ensemble technique
- Process here is.
 - Select **one-data point** as a **root** and **construct a tree**.
 - Select **another data** point as a **root** and **construct another tree**
 - and so on....
 - Until we construct '**m**' **decision trees**



+ Dropout

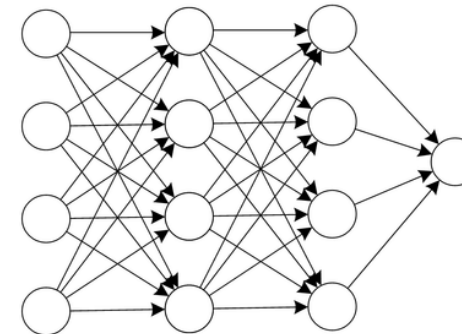
- Q: What's the purpose of Ensemble (Regression–Average Value, classification–Voting) process?
- Answer: To avoid overfitting
- Q: If Overfitting is the biggest problem, then Why not adopt the Ensemble technique for NN to avoid the Overfitting?
- Ans: We cannot adopt the Ensemble technique for NN, because, if we want to try with NN, it will take too much time to train the model,
 - If we want to construct N-number of NN and then train them with different random samples, is almost impossible, because it will take too much time and computation.
 - Also, during Testing, the data will be provided to all Networks to maintain the Ensemble process

+ Dropout

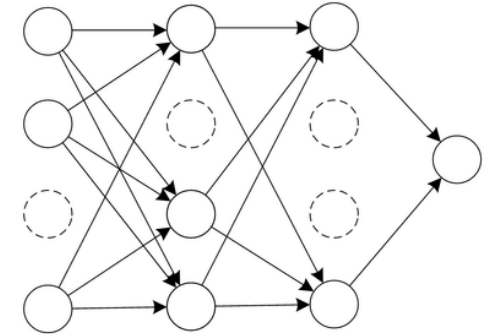
■ Q: What is DROP Out technique?

■ Answer: This technique is used to implement similar idea like Ensemble technique but reduce the training/testing computation and time.

■ In the dropout, we are assigning the probability to each layer including input layer (here's say 0.8) and this is not fixed (you may assign any probability value).



(a) Standard Neural Network

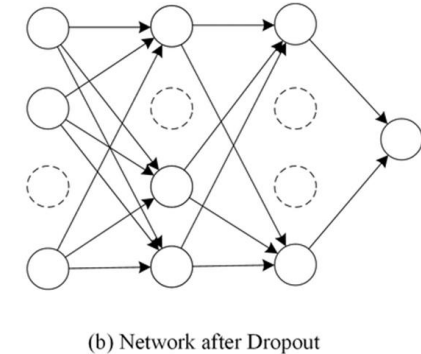
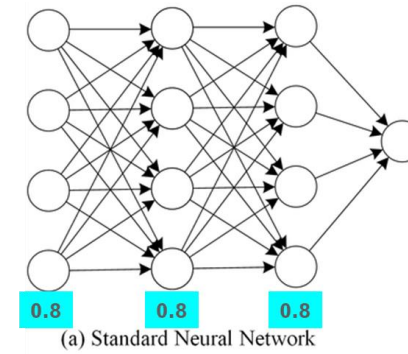


(b) Network after Dropout

Dropout class

```
tf.keras.layers.Dropout(rate, noise_shape=None, seed=None, **kwargs)
```

+ Dropout



- **Q: What's this Probability means here?**
- **Answer:**
- The **Probability 0.8** reflects that the **any neuron** in that layer have **80% chances** to keep in the current iteration while **20% chances** are there to **avoid any neuron**.
- This is done for each neuron **separately in each layer**. The Same process is applicable to the **Input Layers** as well to decide about the **input participation in each iteration**.
- For the **2nd iteration**, we will have **different set of neurons** to **consider** and **drop** based on the **probability** values we are randomly selecting neurons.
- The **3rd iteration** will have **different set of neurons**.

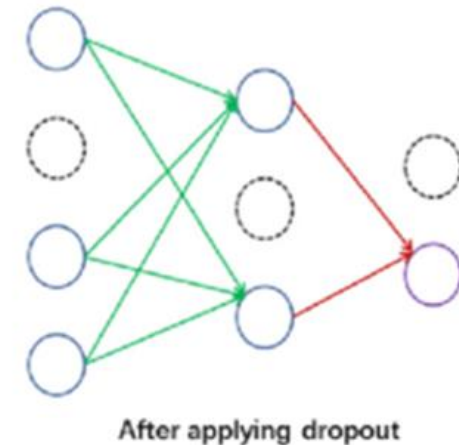
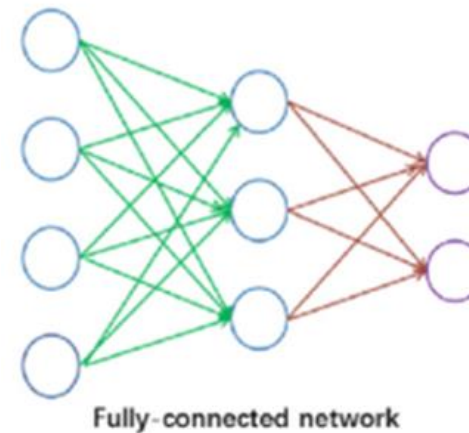
+ Dropout

■ Q: What happen with these dropping and selecting neurons?

■ Answer: With dropping some of the neurons, we are forming a **Thinned network** in 1st iteration, while in 2nd iteration, we are forming different **Thinned network** and so on.

■ Q: How many thinned network are possible here?

■ Answer: In a neural network, if we have N-nodes, then possible **2^n** Thinned networks are possible.



+ Dropout

- In a normal DNN, each neuron are totally based on the inputs received from the previous neurons and then it process the data and provide output which is subsequently input to other neurons. Through this CO-Adoption process, the model is learning a lot and become more specific or sensitive to the data and hence we are getting Overfitting issue.
- DNN with Dropout option, the participation of the neuron is based on the probability values, and we are dropping some of the neurons for that specific iteration means we are restricting neurons to participate in updating their depending weights.
- Hence some of the neuros are not working on CO-Adoption and we break the CO-Adaptation, and make the model more generalize

Basics for CNN Model

Documentation https://keras.io/api/layers/pooling_layers/

Example

+ Convolution Operation on Images

90	83	42	24	2	4
54	87	20	55	8	91
97	15	34	64	53	30
3	29	78	53	53	55
72	27	54	68	9	64
14	25	31	40	71	93
54	46	16	76	67	24

Weight matrix , Filter , Kernel

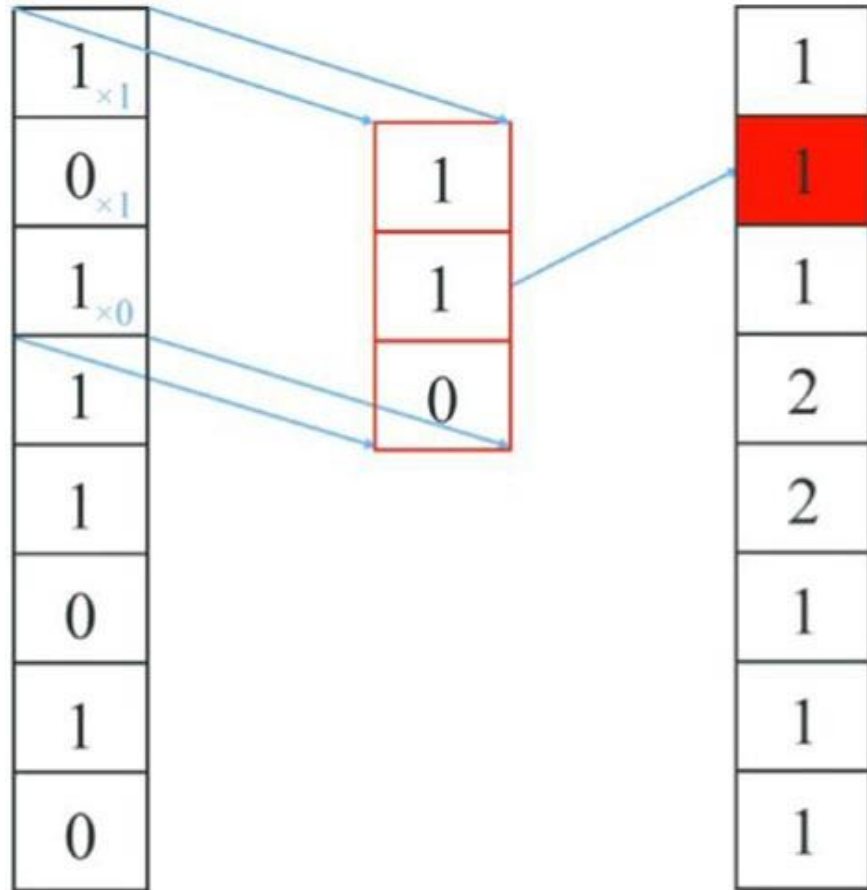
1	0.3
---	-----

33

$$90 * 1 + 83 * 0.3$$

114.9	95.6	49.2	24.6	3.2	4
80.1	93	36.5	57.4	35.3	91
101.5	25.2	53.2	79.9	62	30
11.7	52.4	93.9	68.9	69.5	55
80.1	43.2	74.4	70.7	28.2	64
21.5	34.3	43	61.3	98.9	93

+ Convolution Operation on Images



Conv1D filter

$1_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$	0	0
$0_{\times 0}$	$1_{\times 1}$	$1_{\times 0}$	1	0
$0_{\times 1}$	$0_{\times 0}$	$1_{\times 1}$	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Conv2D filter

+ Edge Detection in Images

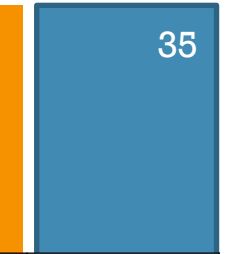
50	50	50	0	0	0
50	50	50	0	0	0
50	50	50	0	0	0
50	50	50	0	0	0
50	50	50	0	0	0
50	50	50	0	0	0

x

1	0	-1
1	0	-1
1	0	-1

=

0	150	150	0
0	150	150	0
0	150	150	0
0	150	150	0



Through this filter we are identifying **Vertical Edges**

For **Horizontal Edges**

1	1	1
0	0	0
-1	-1	-1

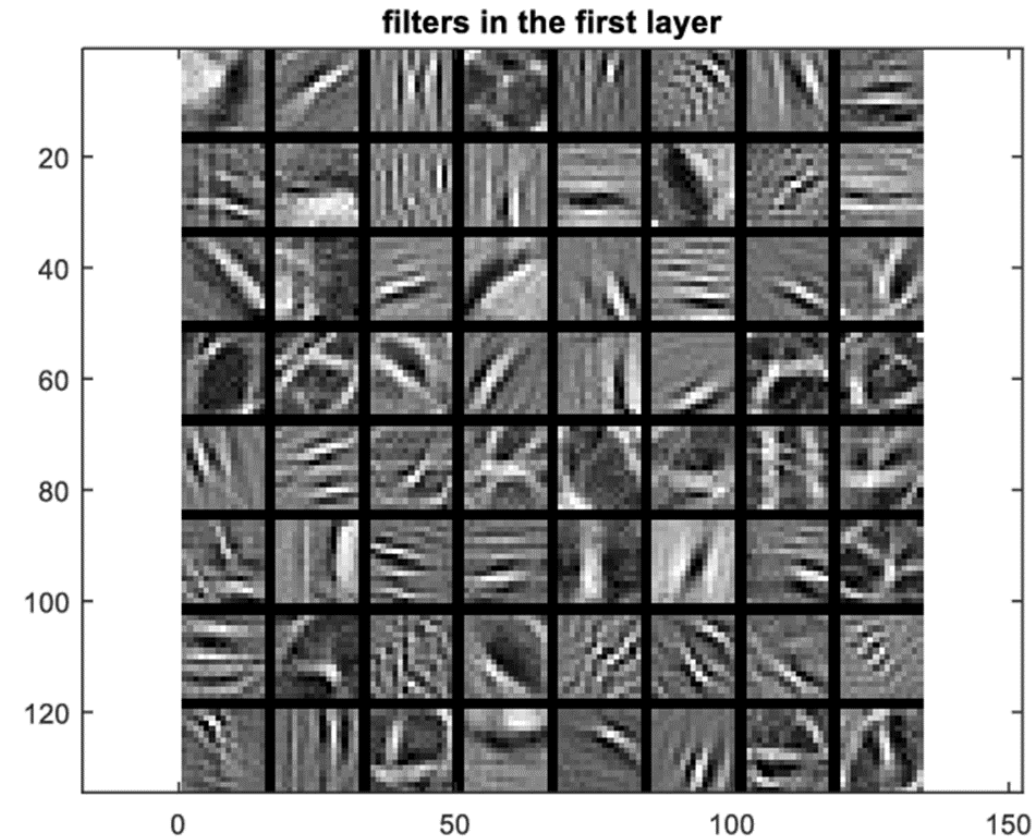
+ Edge Detection in Images - Examples



Vertical edges



Horizontal edges



Various kind of filters
to detect edges

+ Edge Detection in Images

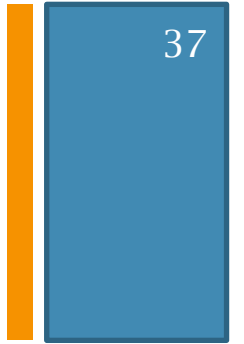
Image

$\times$

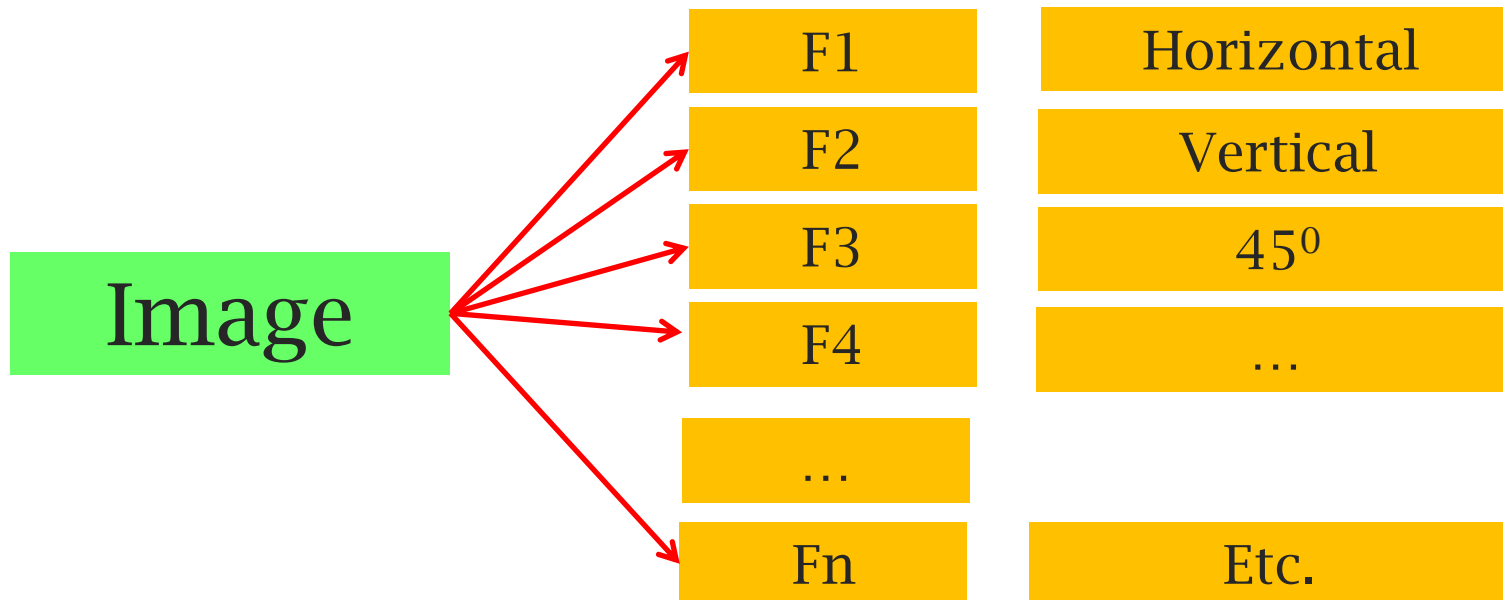
W1	W2	W3
W4	W5	W6
W7	W8	W9

=

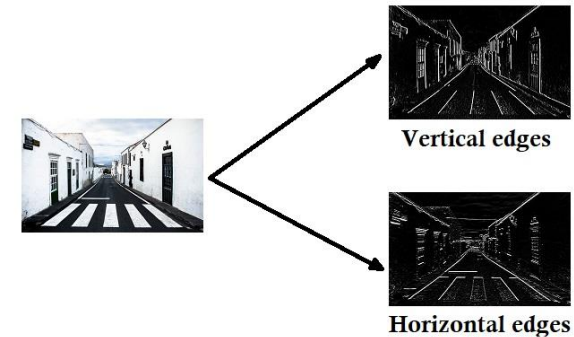
New Image



W's are identify by the CNN



This is one Layer, F's = W's

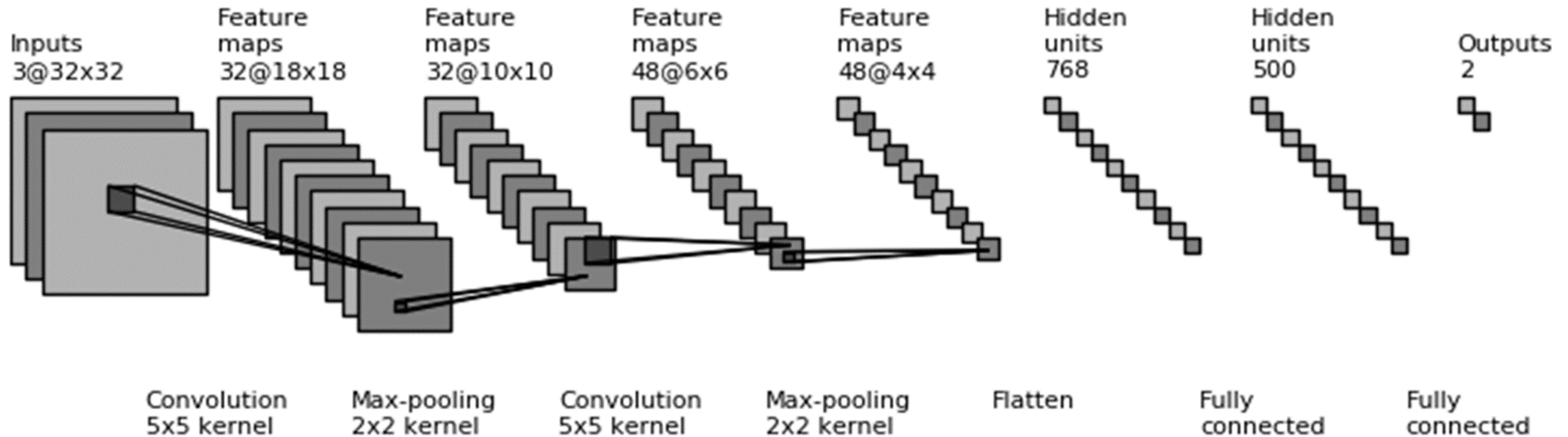


Second Layer

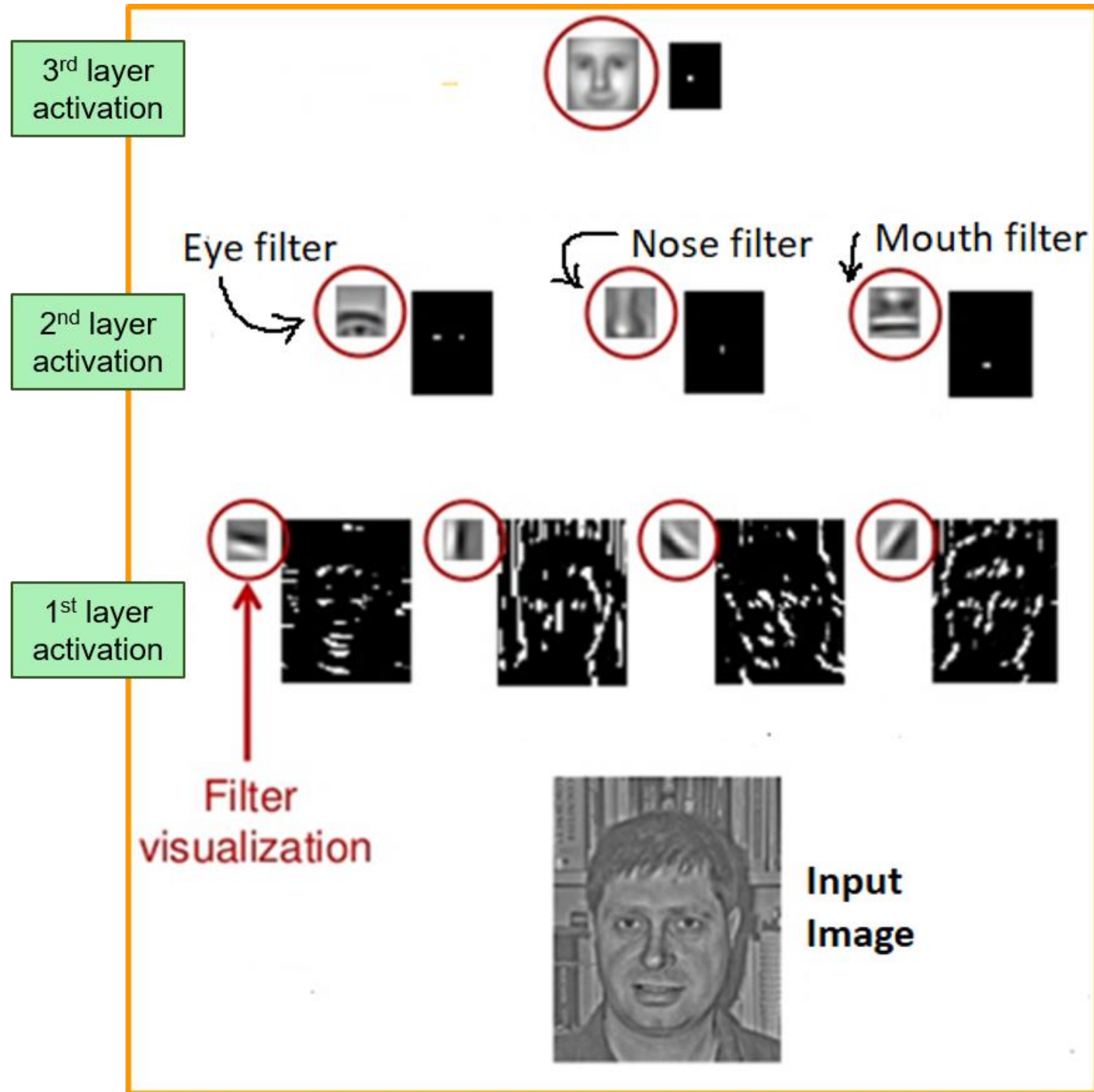
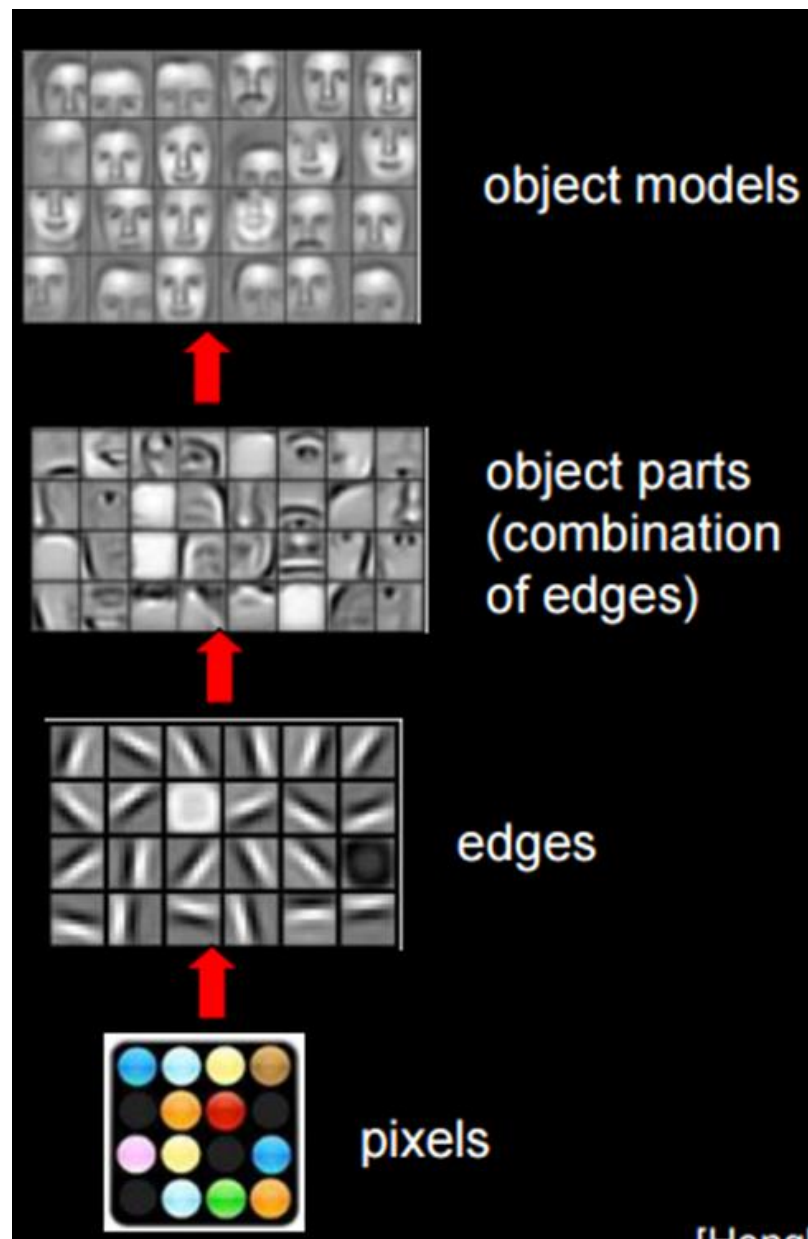
The 2nd Layer will get the input from 1st layer

+ Filter Processes at various layers

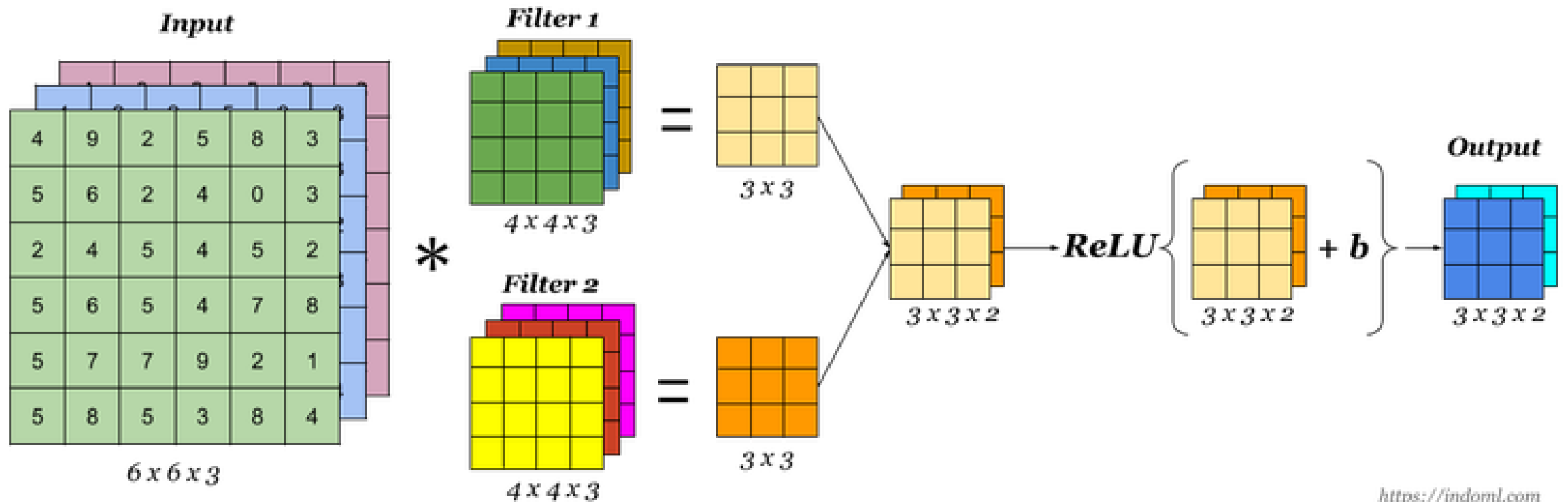
38



+ Filters in CNN

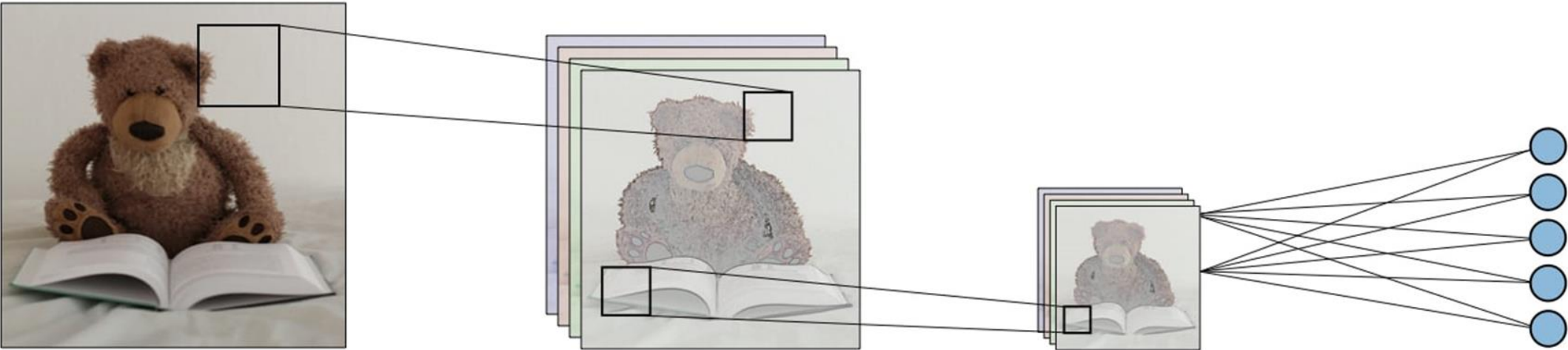
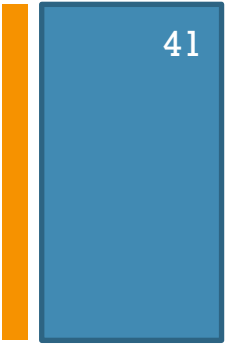
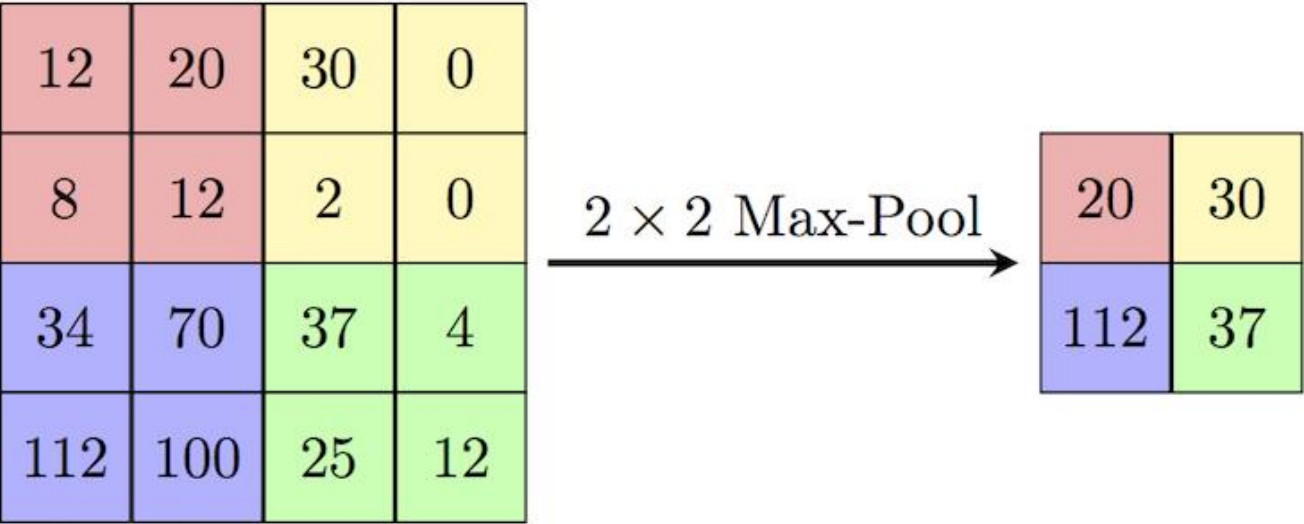


+ Convolution of Color Images with 2 filters



<https://indoml.com>

+ Max Pool



Input image

Convolutions

Pooling

Fully Connected

+ Max Pool (Advantages)

Translation Invariance



Rotation/Viewpoint Invariance



Size Invariance



Illumination Invariance



Matt Krause
mattkrause

5

5

After applying the
MAX-POOL, we are
getting same output

5

0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0
0	255	0	0	0	0

0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0
0	0	0	255	0	0

This can also help us, if
there's invariant images
in Testing data.

We are losing Temporal
information with MAX-
POOL

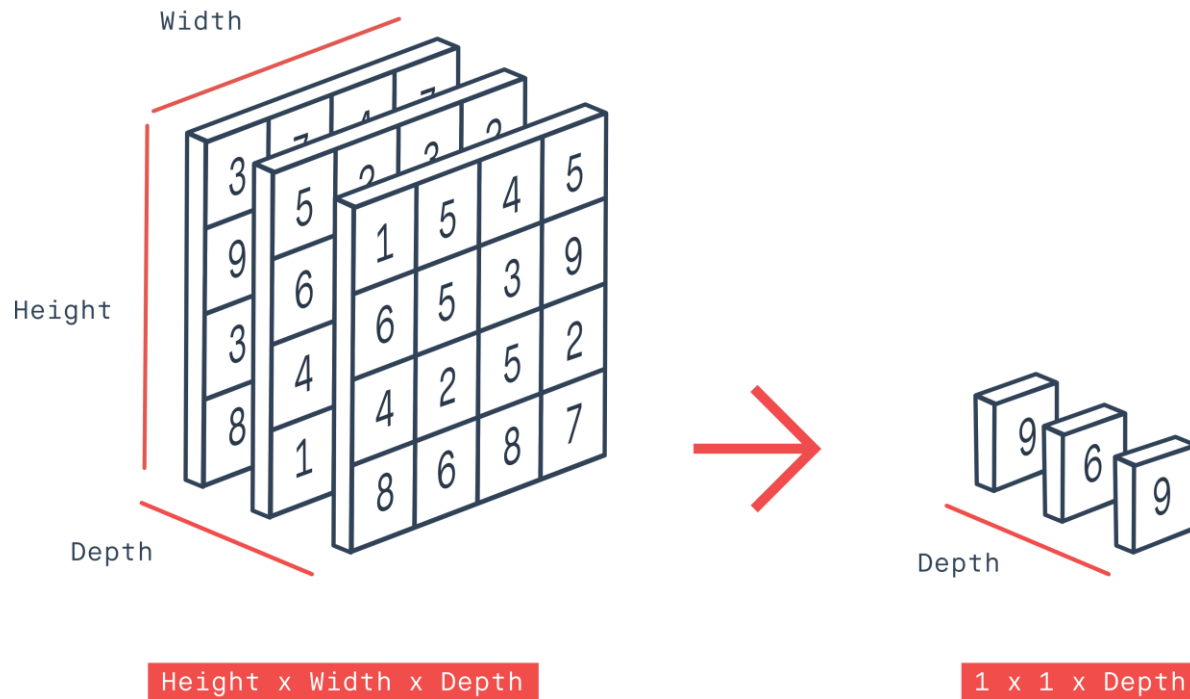


Invariance Problem

+ Global Max Pool

■ Output

- The output is thus a 1-dimensional tensor of size (input channels).

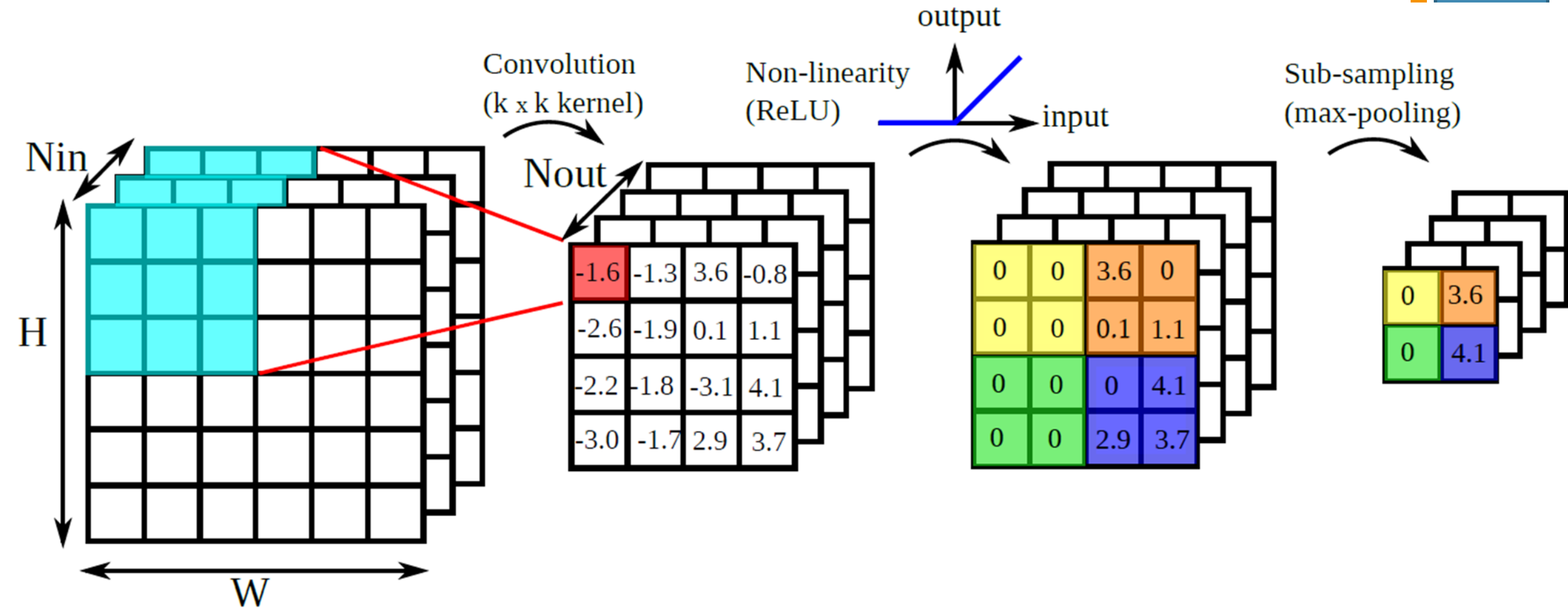


When to use global max pooling

Use **2D Global max pooling** blocks as an alternative to the Flattening block after the last pooling block of your convolutional neural network.

+ Non-Linearity of CNN

44



Convolutional Neural Network (CNN)

Documentation https://keras.io/api/layers/convolution_layers/convolution2d/

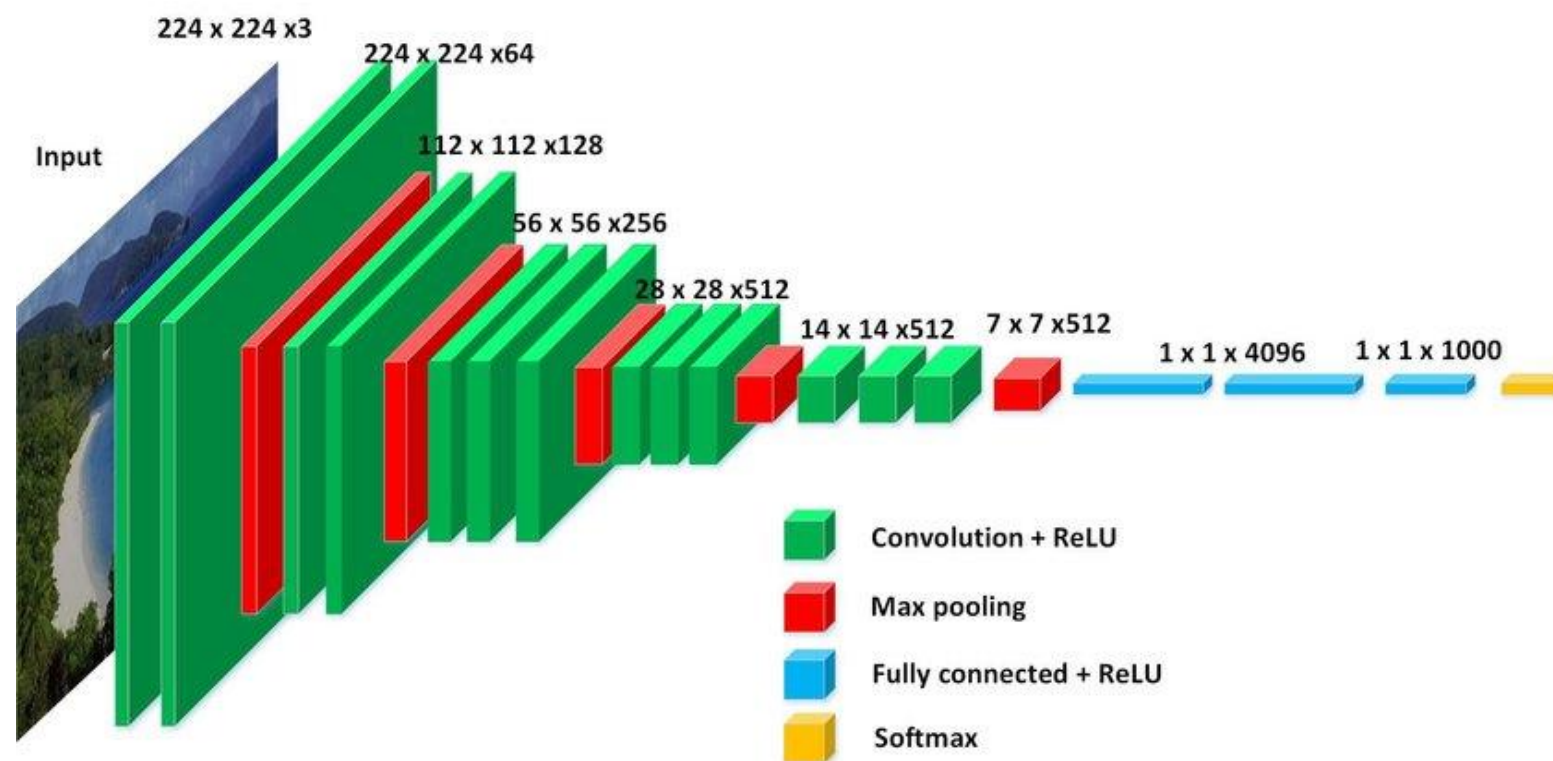
Example <https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529#:~:text=Examples%20of%20CNN%20in%20computer,i.e%20C%20weights%2C%20biases%20etc.>

<https://www.tensorflow.org/tutorials/images/cnn>

+ Convolutional Neural Network

46

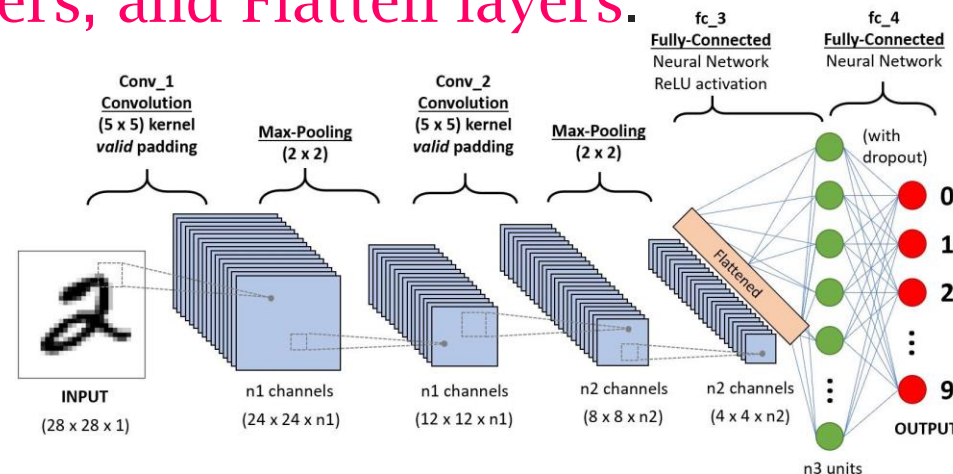
- A CNN is a combination of two components: a feature extractor module followed by a trainable classifier.
- The first component includes a stack of convolution, activation, and pooling layers.
- A DNN does the classification. Each neuron in a layer is connected to those in the next layer.



+ Convolutional Neural Network

47

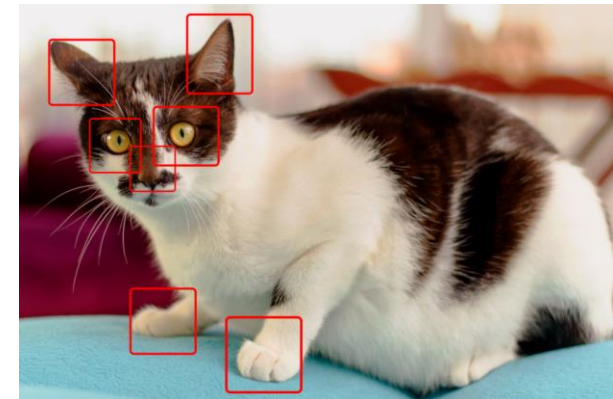
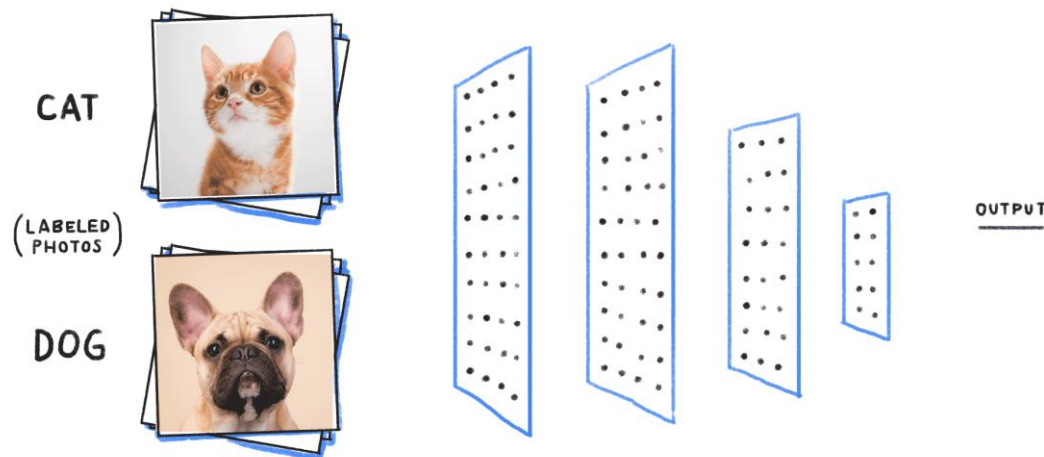
- Today, the overall **architecture** of the CNNs is already streamlined.
- The final part of **CNNs** is very **similar** to **feedforward** neural networks, where there are **fully** connected layers of neurons with weights and biases.
- Just like in **feedforward** neural networks, there is a **loss function** (e.g., **crossentropy**, **MSE**), a **number of activation functions**, and an **optimizer** (e.g., **SGD**, **Adam optimizer**) in CNNs. **Additionally, in CNNs, there are also Convolutional layers, Pooling layers, and Flatten layers.**



+ Why CNN, why not Feedforward Network

48

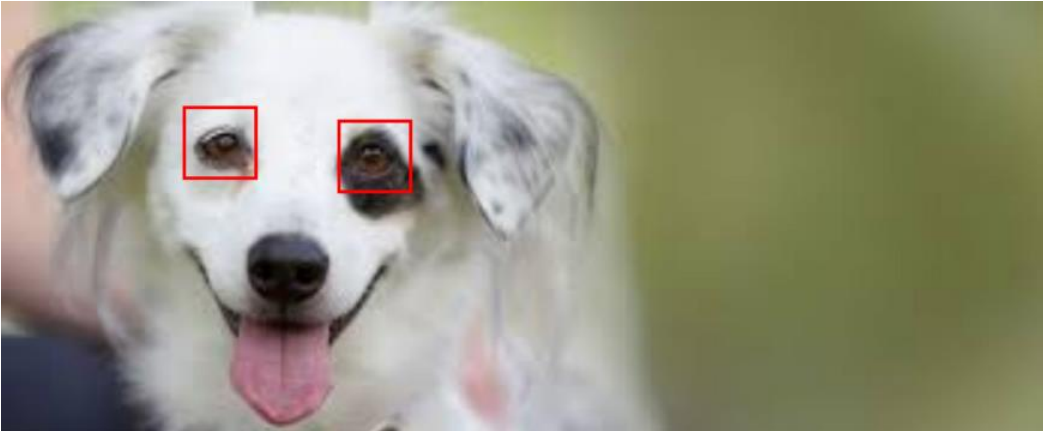
- This is our image; the goal of our network will be to determine whether this image is a cat or not.
- **Dense Layer:** A dense layer will consider the ENTIRE image. It will look at all the pixels and use that information to generate some output.
- **Convolutional Layer:** The convolutional layer will look at specific parts of the image. In this example let's say it analyzes the highlighted parts below and detects patterns there.



+ Why CNN, why not Feedforward Network

49

- Densely connected network has only recognized patterns globally it will look where it thinks the eyes should be present. Clearly it does not find them there and therefore would likely determine this image is not a dog. Even though the pattern of the eyes is present, it's just in a different location.

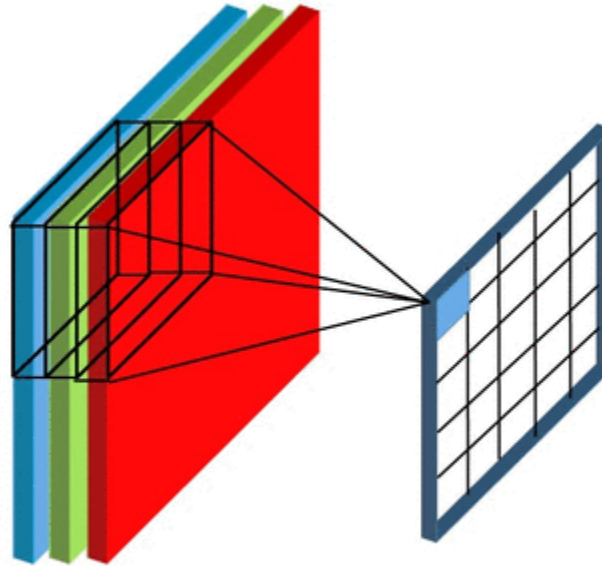
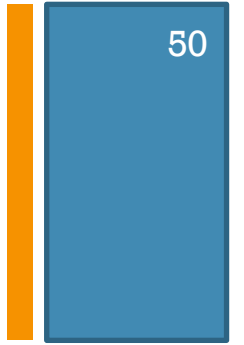


Let's say it's determined that an image is likely to be a dog if an eye is present in the boxed off locations of the image above.

Now let's flip the image.

+ Convolutional Neural Network – How it work

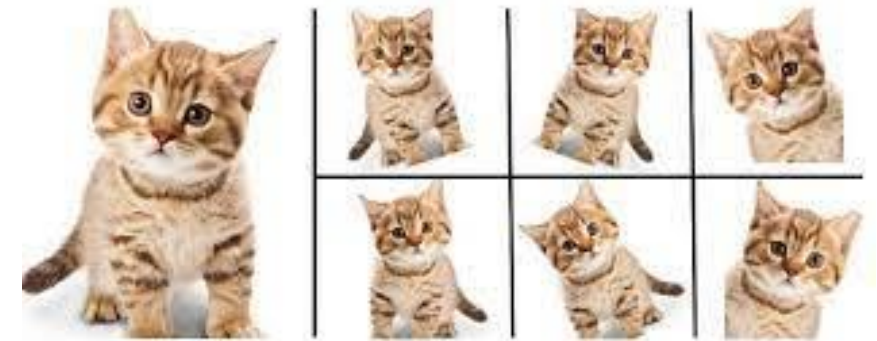
- Since convolutional layers learn and detect patterns from different areas of the image, they don't have problems with the location of the object.
- They know what an eye looks like and by analyzing different parts of the image can find where it is present.



+ Data Augmentations

■ What is Data Augmentation?

- A regularization technique, whereby the dataset is expanded by the creation of artificial variations such as zooming, rotation, shifting, salt/pepper noise, blur etc.
- The augmentation approach chosen is often more important than the type of network architecture used.
- Simple data augmentation like rotation, flip, distortion, obstruction, etc. has proven to improve generalization performance, a simple fact that adding more variation of the same image makes it difficult for the model to memorize, i.e.,



Enlarge your Dataset

OVERFITTING.

Batch Size, Epoch

+ Batches

53

- Loading the data set to the memory we have two options -
 1. You can either load the whole data set to the memory at once
 2. You can load a sample set of data into the memory
- While training a neural network, instead of sending the *entire input in one go*, we *divide* in input into *several chunks* of *equal size randomly*.

Number of batches * Number of images in a single batch = Total number of data set

Example

- Image data set containing *100000 images*, we can convert it into *3125* batches where each batch has *32* images in it.
- Total images = *100000*
- 1 batch = *32* images
- So, a total of 3125 batches, (*3125 * 32 = 100000*).

+ Batch Size

54

- A training dataset can be divided into one or more batches.
 - Batch Gradient Descent. Batch Size = Size of Training Set
 - Stochastic Gradient Descent. Batch Size = 1
 - Mini-Batch Gradient Descent. $1 < \text{Batch Size} < \text{Size of Training Set}$
 - In the case of mini-batch gradient descent, popular batch sizes include 32, 64, and 128.
- **What if the dataset does not divide evenly by the batch size?**
 - It simply means that the *final batch* has *fewer samples* than the *other batches*.
 - Alternately, you can *remove some samples* from the *dataset* or *change* the *batch size* such that the number of samples in the dataset *does divide evenly by the batch size*.

+ Epochs

55

- **One Epoch** is when a **complete dataset** is passed **forward** and **backward** through the neural network **only ONCE**.
- Since **one epoch** is too big to feed to the computer at once we **divide** it in **several smaller batches**.
- **Why more than one epoch?**
 - Updating the ***weights with single pass or one epoch is not enough***.
 - One epoch leads to ***under-fitting*** of the curve.
 - As the no. of epochs ***increases***, more ***no. of times the weight*** are changed in the neural network and the curves goes from ***under-fitting to optimal to overfitting curve***
- The **number of epochs** is traditionally **large**, often **hundreds** or **thousands**, allowing the learning algorithm to run until the error from the model has been sufficiently minimized.
- Number of epochs in the literature set to 10, 100, 500, 1000, and larger

+ Batches

56

- Iterations is the number of batches needed to complete one epoch.
- The number of batches is equal to number of iterations for one epoch.

ACTIVITY

- Assume the size of the dataset is *2000 examples* and we need divide into batches of *500*. then what will be
 - 1. Batch Size : _____
 - 2. Number of Batches: _____
 - 3. Number of Iteration: _____

Transfer Learning

Documentation

https://keras.io/guides/transfer_learning/

<https://towardsdatascience.com/a-comprehensive-hands-on-guide-to-transfer-learning-with-real-world-applications-in-deep-learning-212bf3b2f27a>

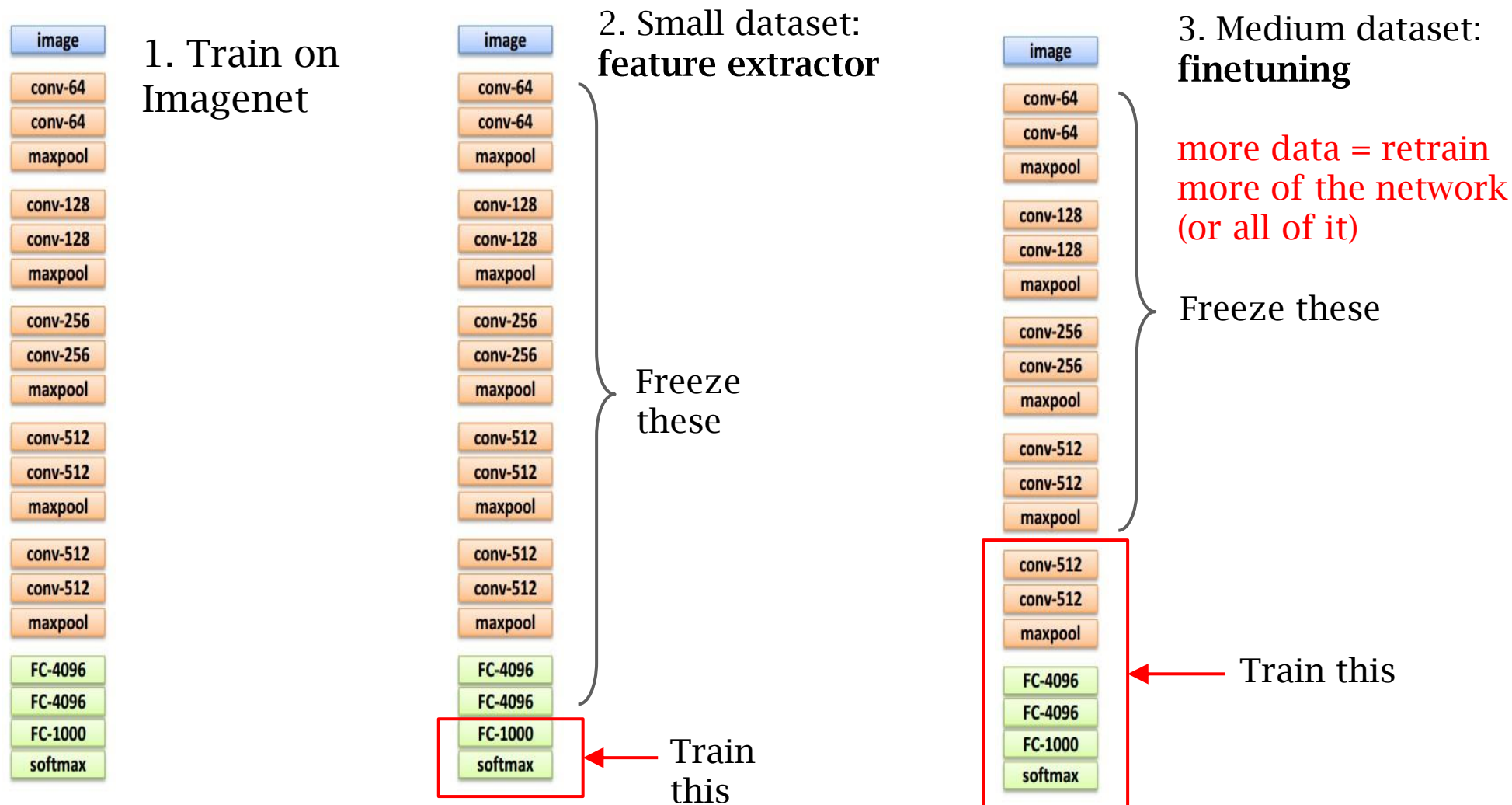
Example

https://www.tensorflow.org/tutorials/images/transfer_learning

<https://www.analyticsvidhya.com/blog/2021/11/transfer-learning-with-tensorflow/>

+ Transfer Learning

58



+ Transfer Learning

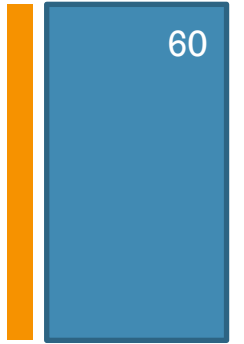
59

- What Is Transfer Learning?
- In transfer learning, the knowledge of an already trained machine learning model is applied to a different but related problem.
- The general idea is to use the knowledge of a model (that learned from a task with a lot of available labeled training data) in a new task that doesn't have much data.
- Instead of starting the learning process from scratch, we start with patterns learned from solving a related task.



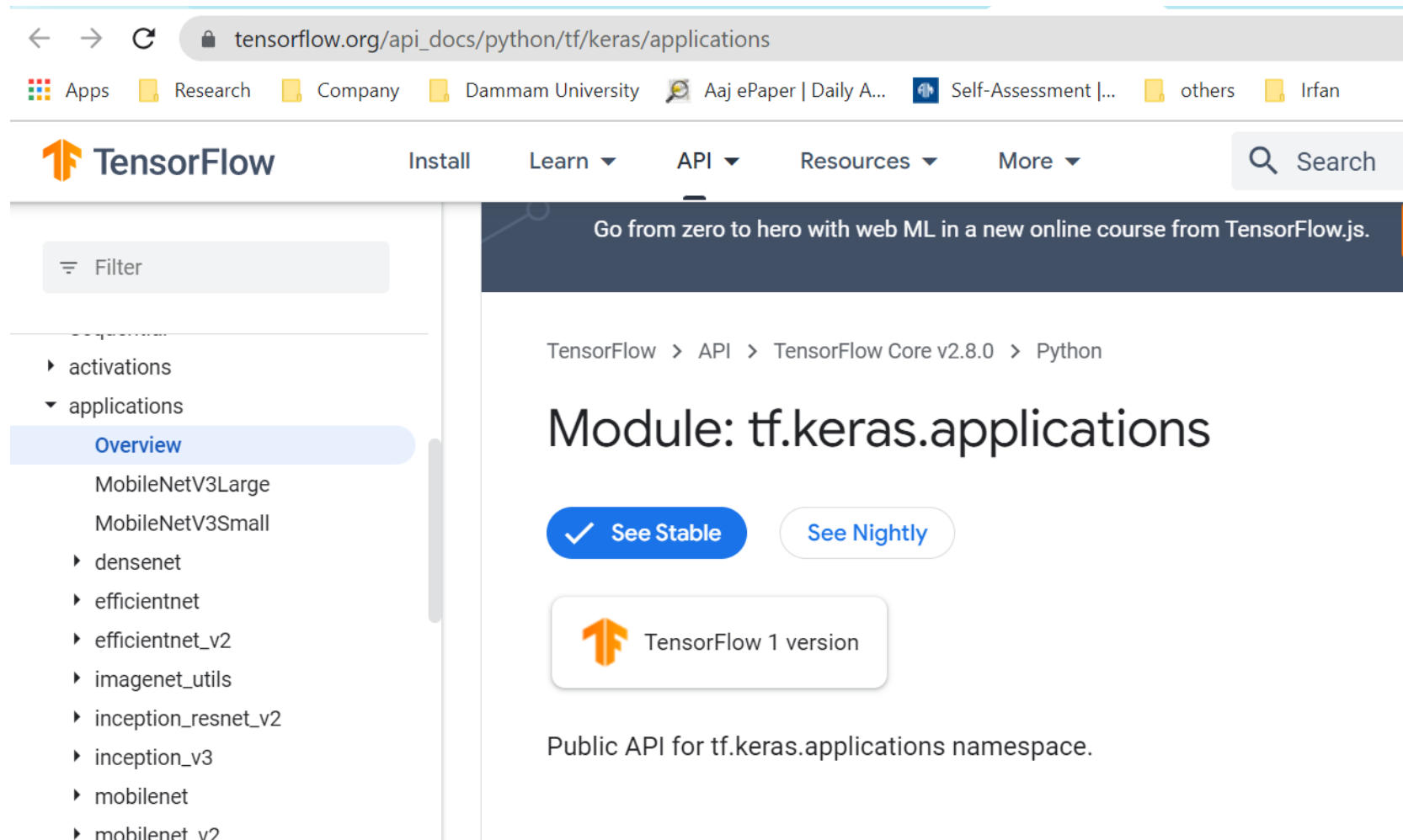
+ How Transfer Learning Works

- In neural networks dealing with images, usually try to detect edges in the earlier layers, shapes in the middle layer and some task-specific features in the later layers.
- In transfer learning, the early and middle layers are used, and we only retrain the task-specific layers. It helps influence the labeled data of the task it was initially trained on.
 - The task 1 and task 2 must have the same setting for input.



+ Popular Transfer Learning Models

- The popular transfer learning models using TensorFlow available on.



The screenshot shows the TensorFlow API documentation page for the `tf.keras.applications` module. The browser address bar displays `tensorflow.org/api_docs/python/tf/keras/applications`. The page features a navigation sidebar on the left with a 'Filter' button and a list of modules including `activations`, `applications` (selected), `MobileNetV3Large`, `MobileNetV3Small`, `densenet`, `efficientnet`, `efficientnet_v2`, `imagenet_utils`, `inception_resnet_v2`, `inception_v3`, `mobilenet`, and `mobilenet_v2`. The main content area has a dark blue banner at the top with the text 'Go from zero to hero with web ML in a new online course from TensorFlow.js.' Below this, the breadcrumb trail is 'TensorFlow > API > TensorFlow Core v2.8.0 > Python'. The title 'Module: tf.keras.applications' is prominently displayed. Two buttons, 'See Stable' (with a checkmark) and 'See Nightly', are provided. A 'TensorFlow 1 version' button is also visible. The description states: 'Public API for tf.keras.applications namespace.'

Recurrent Models

Documentation https://keras.io/api/layers/recurrent_layers/

Example <https://www.tensorflow.org/guide/keras/rnn>
<https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>

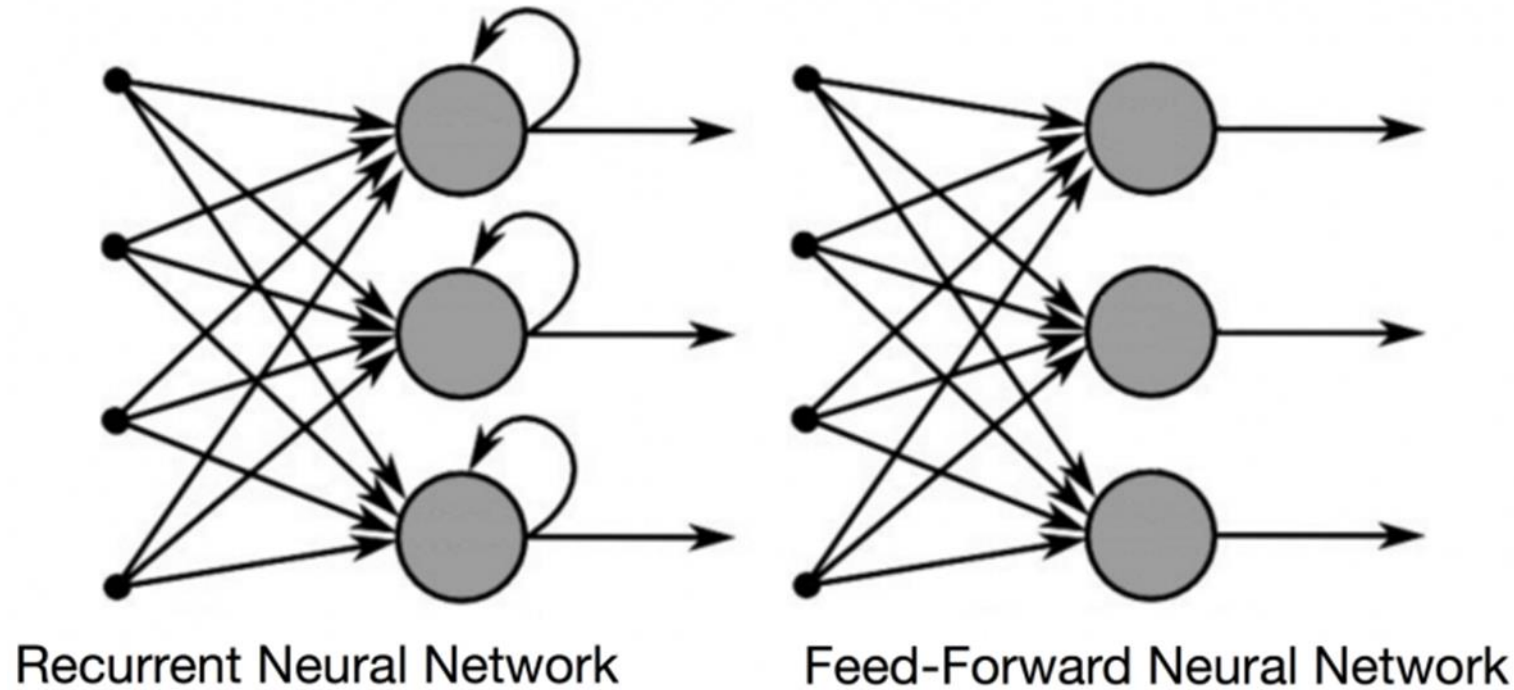
+ Recurrent Neural Network (RNN)

63

- A **Recurrent Neural Network (RNN)** is derived from feedforward networks and give a temporal dynamic behavior (i.e., deal with sequential data).
- It works similarly to human brains to deliver predictive results.
- RNN looks quite similar to a traditional NN, except that a memory-state is added to the neurons.

+ How Recurrent Neural Networks Work?

64



- Let's give word "neuron" as an input to FF-NN and it processes the word character by character.
- When it reach to "r," it has already forgotten about "n," "e" and "u," which makes it impossible for this type of NN to predict which character would come next.

+ How Recurrent Neural Networks Work?

65

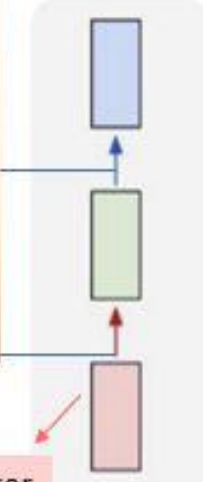
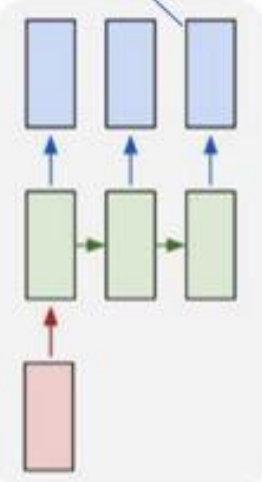
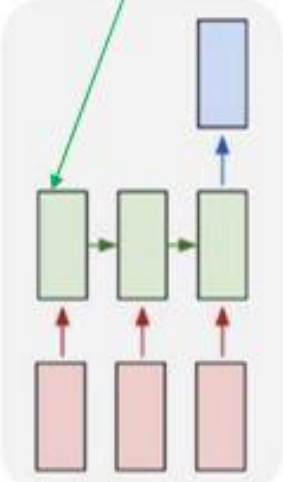
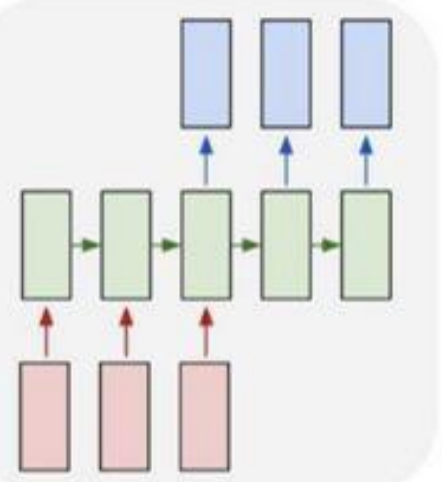
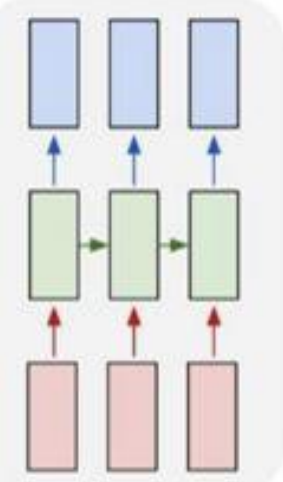
- RNN is able to remember those characters because of its internal memory.
- It produces output, copies that output and loops it back into the network.

Simply: RNN add the immediate past to the present.

- Therefore, a RNN has two inputs: the present and the recent past.
- This is important because the sequence of data contains crucial information about what is coming next, which is why a RNN can do while other algorithms can't.

Types Of RNN Models

66

Type:	One to One	One to Many	Many to One	Many to Many	
Output:	Classified Images	Sentence of words describing image	Sentiment of input	Translation of the language (Machine Translation)	Video with caption
Functions: Matrix Multiply					
Input	Random Images	Image	Review/text/tweet	Input Foreign Language	Video
Application	Basic RNN Use Cases	Image Captioning	Sentiment Analysis	Neural Machine Language Translation	Subtitles for video, Speech recognition
	Vanilla Mode	Sequenced Output	Sequenced Input	Sequenced Input and Sequenced Output (Semi-Synced)	Synced Sequence (Fully-Synced)

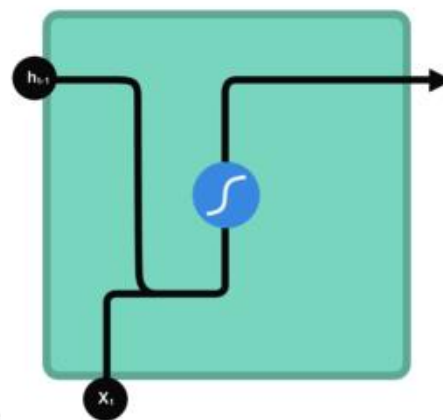
March 2020

+ RNN Cell Structure

67

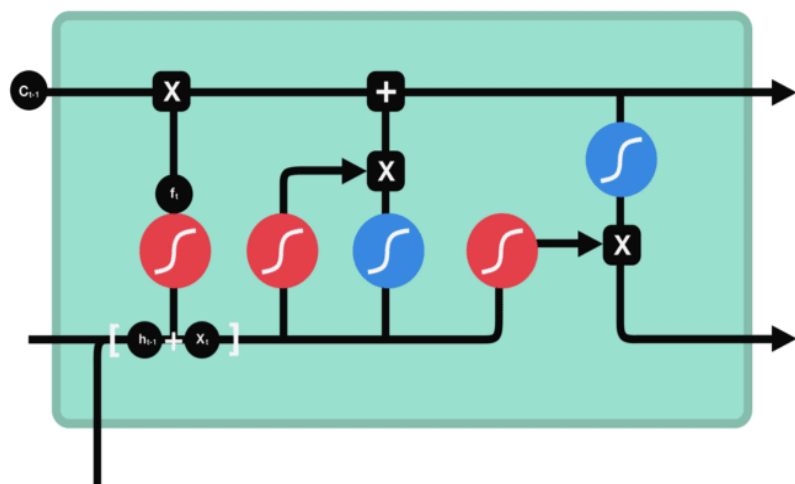
There are 3 types of recurrent neural network:

The simple (RNN)



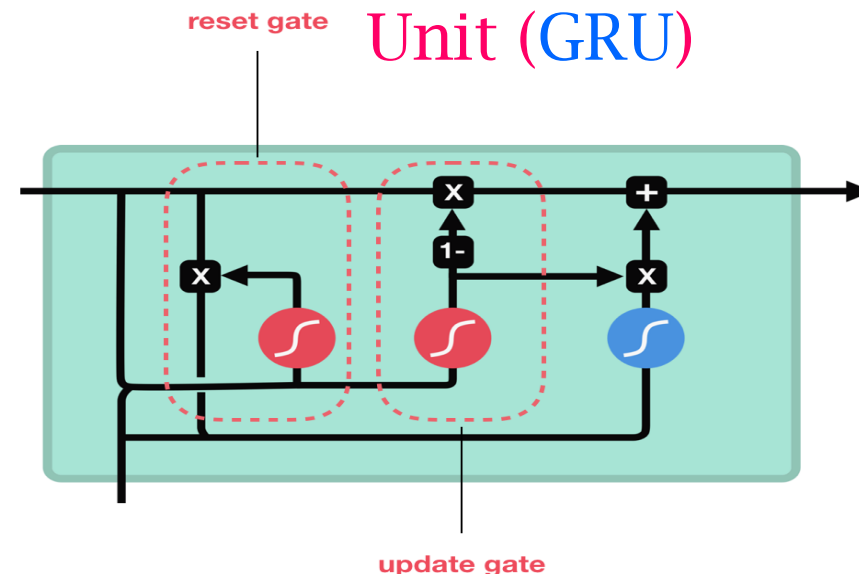
-  Tanh function
- h_t new hidden state
- h_{t-1} previous hidden state
- x_t input
-  concatenation

Long Short Term Memory Unit (LSTM).



- c_{t-1} previous cell state
- f_t forget gate output
- i_t input gate output
- $\tilde{c}_t$ candidate

Gated Recurrent Unit (GRU)





End of Lecture – 08